

FORMAL LANGUAGES AND COMPILERS

prof.s Giovanni Agosta, Luca Breveglieri and Daniele Cattaneo

Exam of **FRIDAY 17 JANUARY 2025 – Theory Part**

WITH SOLUTIONS – FOR TEACHING PURPOSES THE SOLUTIONS ARE WIDELY COMMENTED

LAST NAME (SURNAME) + FIRST NAME:

(capital letters please)

MATRICOLA:

(or PERSON CODE)

SIGNATURE:

TEACHER: ☐ Prof. G. AGOSTA – ☐ Prof. L. BREVEGLIERI – ☐ Prof. D. CATTANEO

INSTRUCTIONS – READ CAREFULLY:

- The exam is open book: textbooks and personal notes are permitted.
- Pencil writing is permitted, except on the cover page (this sheet).
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: lab part 1 h – theory part 2 h.

SCORING AND GRADE ATTRIBUTION:

- The exam is in written form and consists of two parts:

Theory syntax and semantics of languages, divided in four sections: (75% of the final grade)

- 1) regular expressions and finite automata (1 exercise × 10 points)
- 2) syntax analysis and parsing methodologies (1 exercise × 10 points)
- 3) grammar design – translation – semantic analysis (2 exercises × 5 points)
- 4) *bonus* question (1 exercise × 5 points)

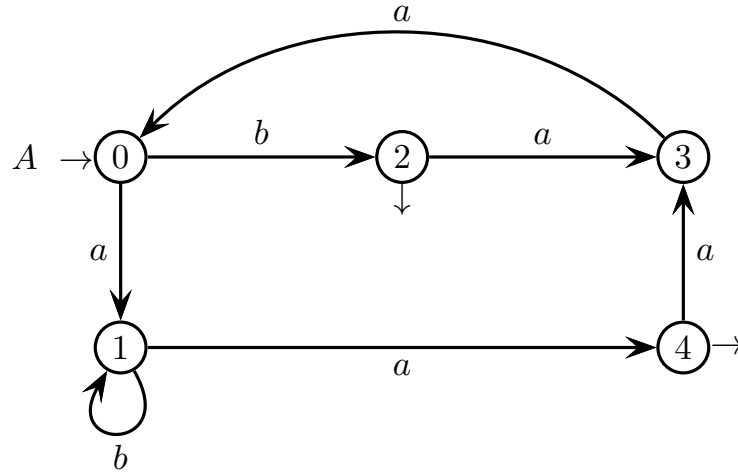
Lab compiler design by Flex and Bison (25% of the final grade)

- 1) basic questions (30 points)
- 2) *bonus* question (5 points)

- For the theory part to be valid, the candidate must achieve a grade of at least 5/10 in each of the three sections 1, 2 and 3. If this is not the case, section 4 is not evaluated. Section 4 has no threshold on its own. Correctly answering all the questions in sections 1, 2 and 3 allows the candidate to achieve a grade of 30/30 (but not *cum laude*) for the theory part.
- For the lab part to be valid, the candidate must achieve a grade of at least 15/30 in the basic questions. The bonus question is evaluated only if the threshold is reached.
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- The final grade is the weighted average of the theory part (75 %) and of the lab part (25 %), and must be $\geq 18/30$ (*cum laude* $\geq 32/30$).

1 Regular expressions and finite automata (10 points)

1. Consider the following deterministic finite-state automaton A , with five states, over the two-letter alphabet $\Sigma = \{a, b\}$:



Answer the following questions (use the figures / tables / spaces on the next pages):

- (a) Is automaton A minimal? If not, minimize it.
- (b) By using the Brzozowski-McCluskey method (BMC), find a regular expression R_A that generates the language $L(A)$ of automaton A .
- (c) Now consider the regular expression R below, over the same two-letter alphabet $\Sigma = \{a, b\}$:

$$R = ([a]baa)^* \quad // \text{ "[]" is the optionality operator}$$

Write all the strings of length four or less (≤ 4) of the language $L(R)$ generated by R .

- (d) By using the Berry-Sethi method (BS), find a deterministic finite-state automaton A_R that recognizes language $L(R)$.
- (e) Find the finite-state automaton that recognizes the mirror language $L(R)^R$. Is this automaton deterministic?

space for questions (a-d)

space for questions (a-d)

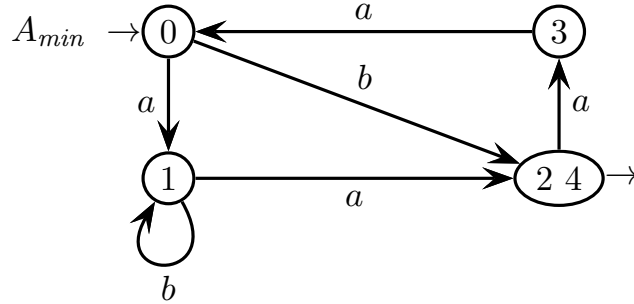
space for questions (a-d)

Solution

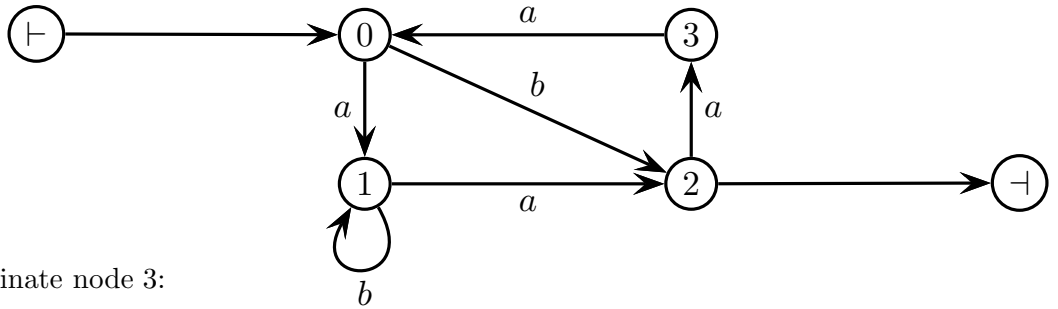
(a) Automaton A is not minimal, as the two final states 2 and 4 are indistinguishable, as both go to the same state with the same transitions. All the other (non-final) states are distinguishable:

- state 0 is distinguishable from 1 because with label b they respectively transition to two distinguishable states (1 itself, and 2 which is final unlike 1),
- state 3 is distinguishable from 0 and 1 because state 3 has a transition with label b while states 0 and 1 do not.

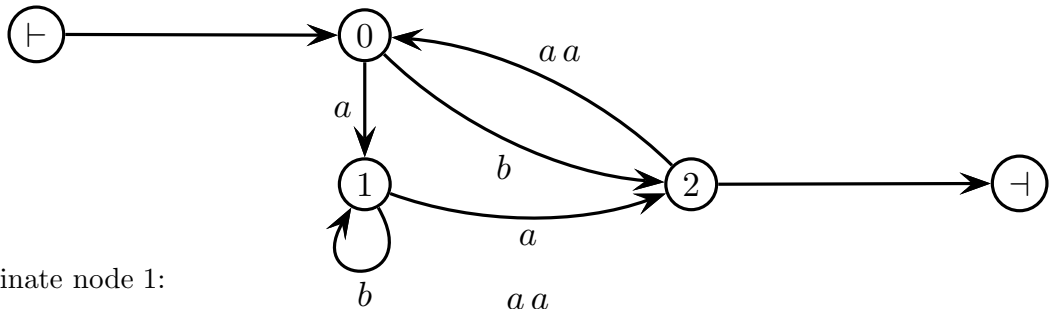
Therefore we can merge states 2 and 4. The minimum automaton A_{min} is:



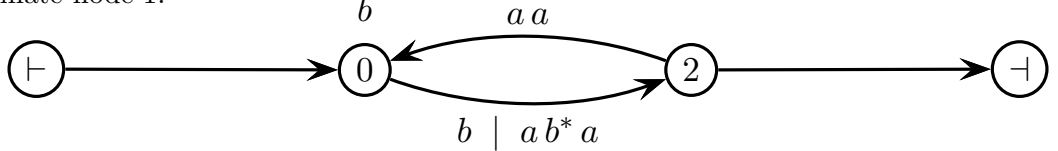
(b) We start from the minimal automaton A_{min} . Normalize initial and final states:



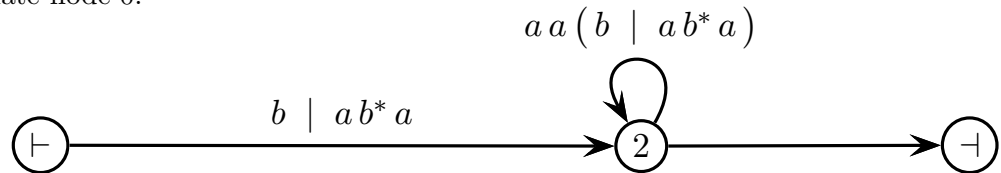
Eliminate node 3:



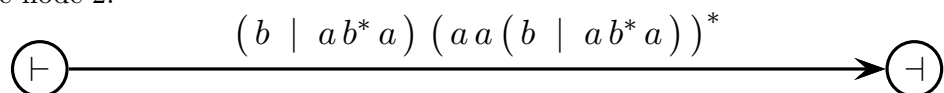
Eliminate node 1:



Eliminate node 0:



Eliminate node 2:



We find that regular expression R_A is:

$$R_A = (b \mid a b^* a) (a a (b \mid a b^* a))^*$$

Alternatively, if node 2 is eliminated before node 0, a different but equivalent regular expression is obtained:

$$R'_A = ((b \mid a b^* a) a a)^* (b \mid a b^* a)$$

Both these regular expressions are shown with the minimum number of parenthesis required for this specific formulation. If the parentheses around the last union operator are left unstated as shown here:

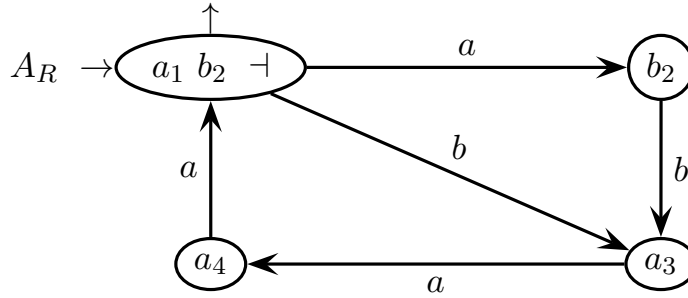
$$\begin{aligned} R'' &= ((b \mid a b^* a) a a)^* b \mid a b^* a \\ &= (((b \mid a b^* a) a a)^* b \mid a b^* a) \\ &\neq R_A \\ &\neq R'_A \end{aligned}$$

the expression becomes incorrect because of the lower precedence of the union operator with respect to the Kleene star.

- (c) The language $L(R)$ contains the following strings: $\{\varepsilon, b a a, a b a a\}$
(d) Berry-Sethi applied to the regular expression R :

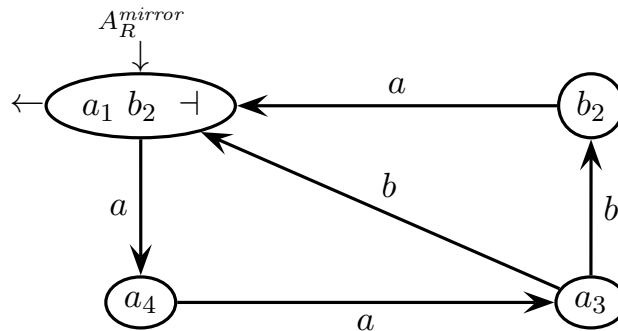
$$R_{\#} = ([a_1] b_2 a_3 a_4)^*$$

initials	a_1	b_2	$\neg$
terminal	followers		
a_1	b_2		
b_2	a_3		
a_3	a_4		
a_4	a_1	b_2	$\neg$

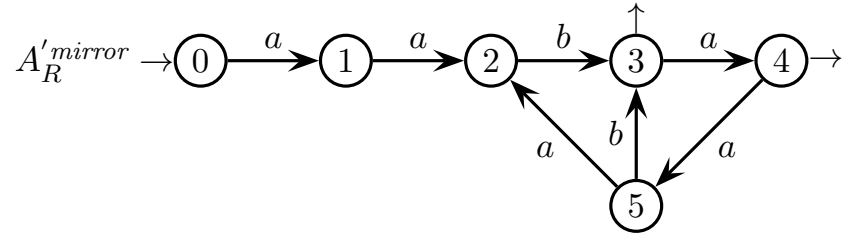


Note that automaton A_R is already minimal.

- (e) The construction of the mirror automaton A_R^{mirror} is as usual, by reversing all the arcs and switching the initial and final states:



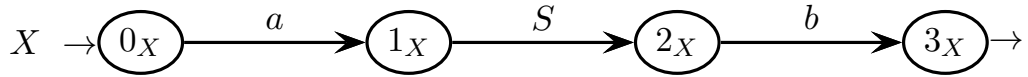
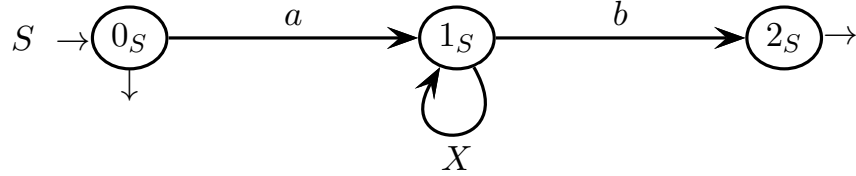
It can be seen however, that now from state a_3 we have two outgoing arcs labeled b , therefore the mirror automaton A_R^{mirror} is nondeterministic. It can be determinized, of course, for example by using Berry-Sethi. Similarly, it is also possible to reverse regular expression R and apply Berry-Sethi to it, obtaining the following deterministic automaton immediately:



This last automaton is not only deterministic but also minimal.

2 Syntax analysis and parsing methodologies (10 points)

1. Consider the EBNF grammar G below, represented as a network of recursive machines, over a two-letter terminal alphabet $\Sigma = \{a, b\}$ and a two-symbol nonterminal alphabet $V = \{S, X\}$ (axiom S):

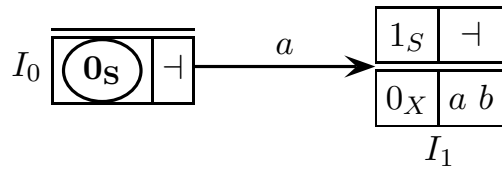


Answer the following questions (use the figures / tables / spaces on the next pages):

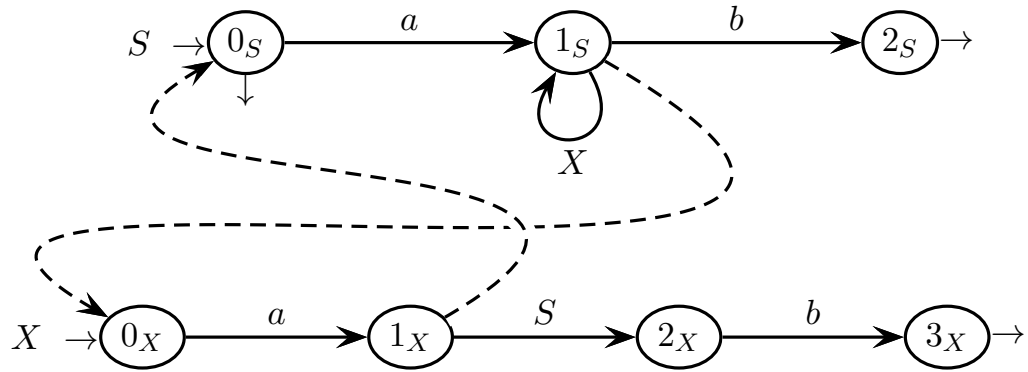
- (a) List all the strings of length six or less (≤ 6) of language $L(G)$ and draw their syntax trees.
- (b) Draw the complete pilot of grammar G and prove that grammar G is of type ELR.
- (c) Compute all the guide sets of grammar G and prove that grammar G is of type ELL, for some $k \geq 1$.
- (d) Write the syntactic procedure of nonterminal S (axiom).
- (e) Say if language $L(G)$ is regular and reasonably explain your answer.
- (f) Suppose nonterminal X is nullable, e.g., by assuming that the net state 0_X is final (besides being initial). Orderly examine whether the new grammar G is ELL, ELR, or ambiguous. Explain your answers.

valid strings of length ≤ 6 and their syntax trees – question (a)

complete pilot of grammar G and conflict analysis – question (b)



guide sets of grammar G and conflict analysis – question (c)

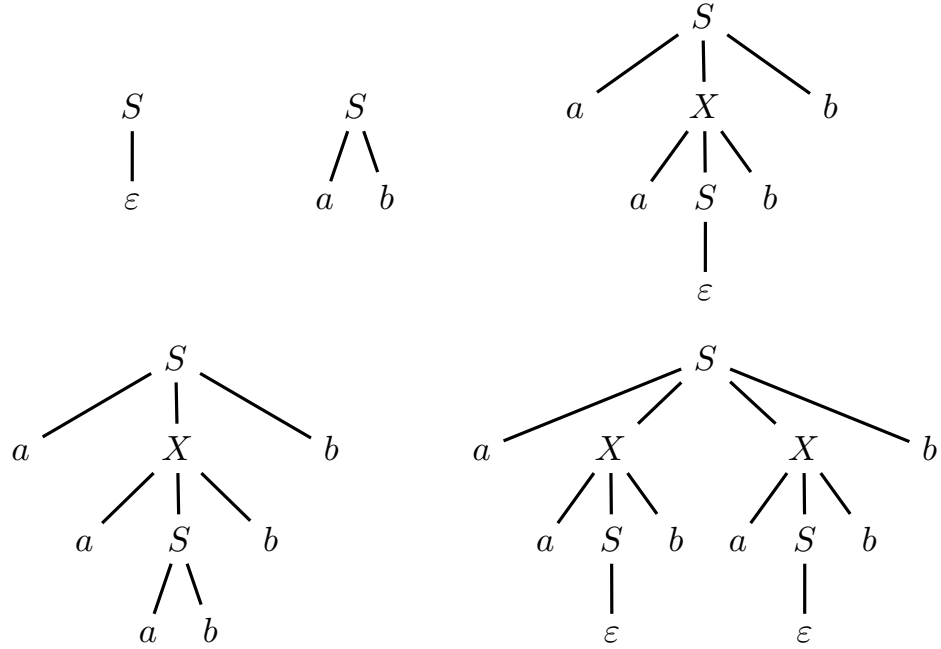


syntactic procedure of nonterminal S (axiom) – question (d)

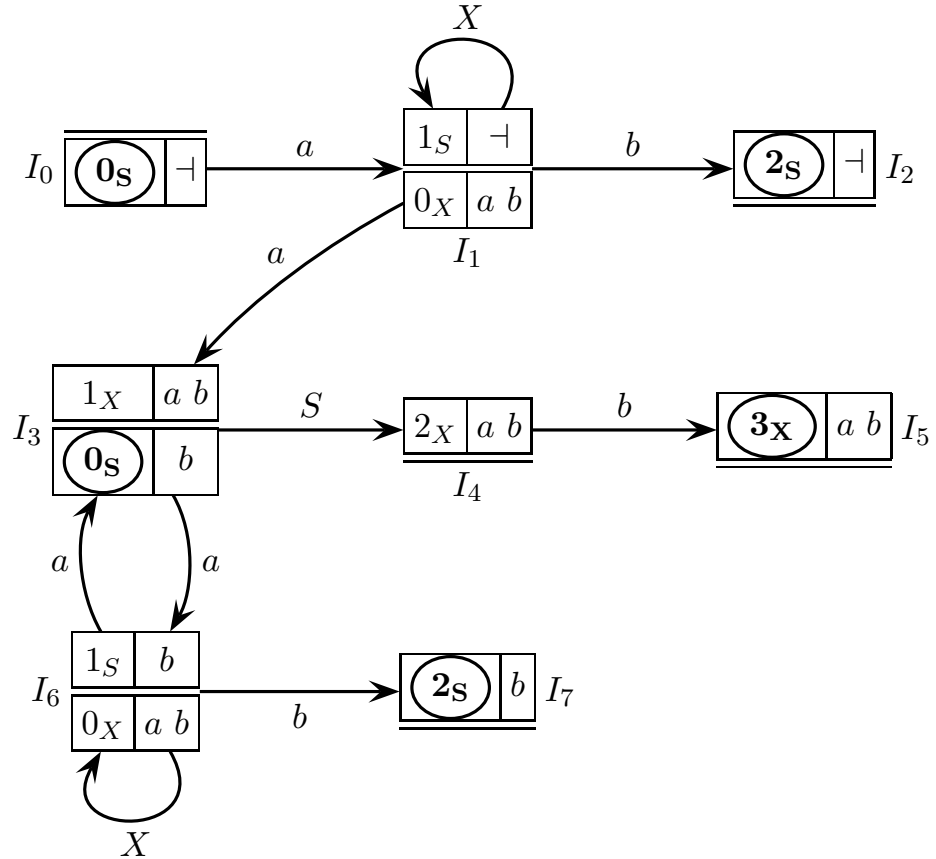
space for questions (e-f-g)

Solution

(a) Strings of length ≤ 6 (five in total): ε , ab , $aabb$, $aaabbb$ and $ababbb$. Trees:



(b) Here is the complete pilot of grammar G :

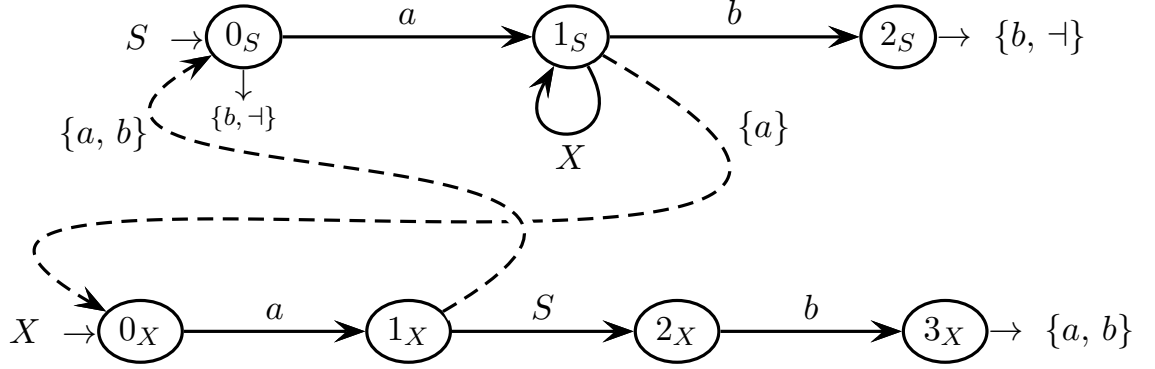


Notice how the closure candidates with state 0_S in I_1 and I_6 have a lookahead including a , as

the initial characters of $L(1_S)$ also include the initial characters of X due to the transition $1_S \xrightarrow{X} 1_S$.

There are no SR, RR or Convergence conflicts, thus grammar G is ELR.

- (c) Here is the PCFG of grammar G , with the guide sets (for $k = 1$):



There are no guide set conflicts on the two bifurcation states 0_S and 1_S , thus grammar G is ELL with $k = 1$.

- (d) Syntactic procedure of nonterminal S (axiom), coded in the standard fully precise way (each transition is checked before being taken and each error stops the parser in the proper state):

<pre> procedure S if $cc \in \{a\}$ then $cc := next$ LOOP: if $cc \in \{a\}$ then call X goto LOOP else if $cc \in \{b\}$ then $cc := next$ if $cc \in \{b, -\}$ then return else error else error else if $cc \in \{b, -\}$ then return else error end procedure </pre>	<pre> // machine S // check 0 // goto 1 // check 1 // call X // stay 1 // check 1 // goto 2 // check and exit 2 // stop 2 // stop 1 // check and exit 0 // stop 0 </pre>
---	--

More elegantly, by using an explicit iterative construct (while-do) instead of a goto label, and more compactly, by grouping all the error cases at the end:

```

procedure  $S$                                      // machine  $S$ 
if  $cc \in \{a\}$  then                                // check 0
     $cc := next$                                        // goto 1
    while  $cc \in \{a\}$  do call  $X$  // check 1, call  $X$  and stay 1
    if  $cc \in \{b\}$  then                                // check 1
         $cc := next$                                        // goto 2
        if  $cc \in \{b, \neg\}$  then return // check and exit 2
    else if  $cc \in \{b, \neg\}$  then return // check and exit 0
    error                                             // stop 0, 1 or 2
end procedure

```

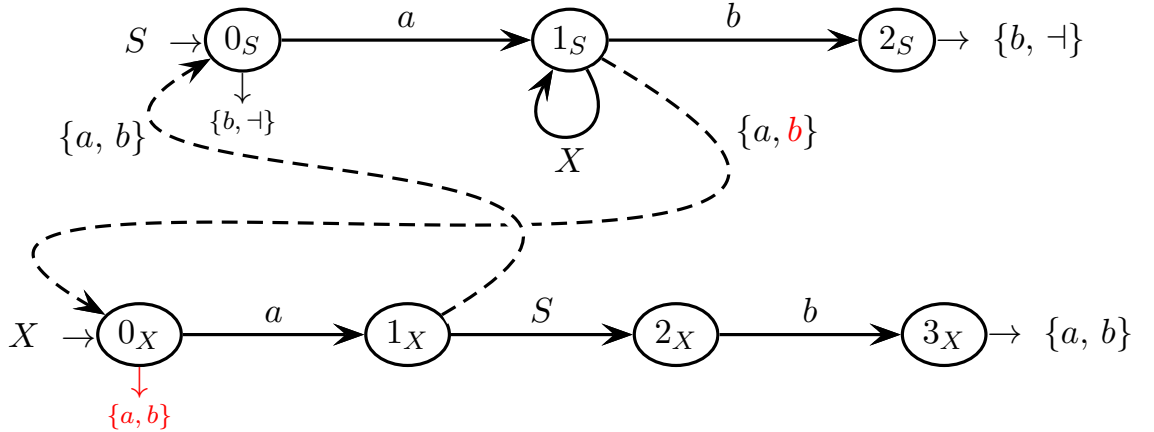
though in this way the localization of the error is less precise.

- (e) Actually $L(G)$ is the Dyck language restricted to have exactly one outermost ab pair:

$$L(S) = \varepsilon \mid (a(L(X))^*b) = \varepsilon \mid (a(aL(S)b)^*b)$$

From this recursive equation, it is possible to have strings with nested and concatenated ab pairs, but there is exactly one outermost pair. The complete Dyck language can instead have also strings like $abab$, etc. Clearly this restriction of the Dyck language is not regular.

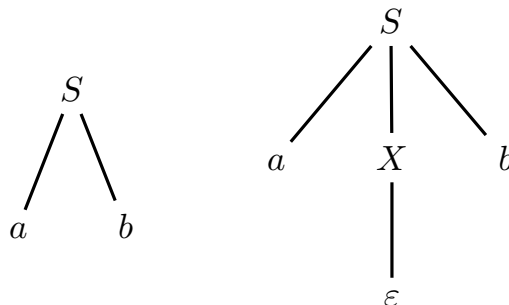
- (f) Under the hypothesis that nonterminal X is nullable, the PCFG of grammar G changes in this way:



Now there are a conflict on b in the net state 1_S and a conflict on a in the net state 0_X . Thus the new grammar G is no longer ELL, for $k = 1$. See below for the case $k > 1$

In the pilot, an SR conflict on b is present in the m-states I_1 and I_6 . Thus the new grammar G is no longer ELR (and consequently not ELL either, though this was already known).

Finally, note that the new grammar G is ambiguous, e.g., string ab has two trees:



Of course, ambiguity implies that the new grammar G is neither ELR nor ELL, though this was already known. It adds however, that the new grammar G is not ELL for any $k \geq 1$.

It is worth repeating that classifying a grammar as not ELL(1) does not imply that it is not ELR(1) or that it is deterministic. On the contrary, an ambiguous grammar cannot be ELL(k) or ELR(k). However, in general the language described by that grammar may still be deterministic (and as a consequence it may admit a different grammar formulation that is indeed ELR(1)).

3 Grammar design – translation – semantic analysis (5 + 5 points)

1. Consider a language for describing a list of *three-dimensional meshes*. A mesh consists of a set of points in three-dimensional spaces, which are the *vertices* for the *triangles* constituting the surface of the mesh. The vertices appear as a list of points, and each vertex is given a unique zero-based numeric ID derived from the vertex position in the list. Thus, the first vertex has an ID of zero, the second vertex has ID 1, the third has ID 2, and so on. These IDs are used as references to the vertices in the description of each triangle.

An example of a string in this language is the following:

```
mesh Pyramid
  vertices (0,0,0) (2,0,0) (1,0,2) (1,2,1)
  triangles (0,1,2) (0,1,3) (1,2,3) (0,2,3)
end

mesh Triangle
  vertices (0,0,0) (1,0,0) (0,1,0)
  triangles (0,1,2)
end
```

In a valid string, there have to be one or more *mesh descriptions*, enclosed within the **mesh** and **end** keywords. Each mesh has a *name*, which is an arbitrary identifier that appears immediately after the **mesh** keyword. Inside each description, there are two *sections*, for **vertices** and **triangles** respectively, introduced by eponymous keywords. Each section contains a non-empty list of three-element integer tuples (hence the coordinates of the vertices are also integers).

Answer the following questions:

- (a) Write an EBNF non-ambiguous grammar that generates the language just described. In the grammar, use the terminals **num** and **id** (not to be further expanded) for integer numbers and arbitrary alphanumeric strings, respectively.
- (b) Draw the syntax tree of the sample string shown below:

```
mesh Triangle
  vertices (0,0,0) (1,0,0) (0,1,0)
  triangles (0,1,2)
end
```

- (c) Extend the grammar, so as that a mesh may optionally contain sub-meshes, as follows:

```
mesh Triangle
  vertices (0,0,0) (1,0,0) (0,1,0)
  triangles (0,1,2)
  submesh
    mesh X vertices (0,0,0) triangles (0,0,0) end
    mesh Y vertices (0,1,0) triangles (0,0,0) end
  end
```

You may just list the modified or new rules.

space for answering questions (a-c)

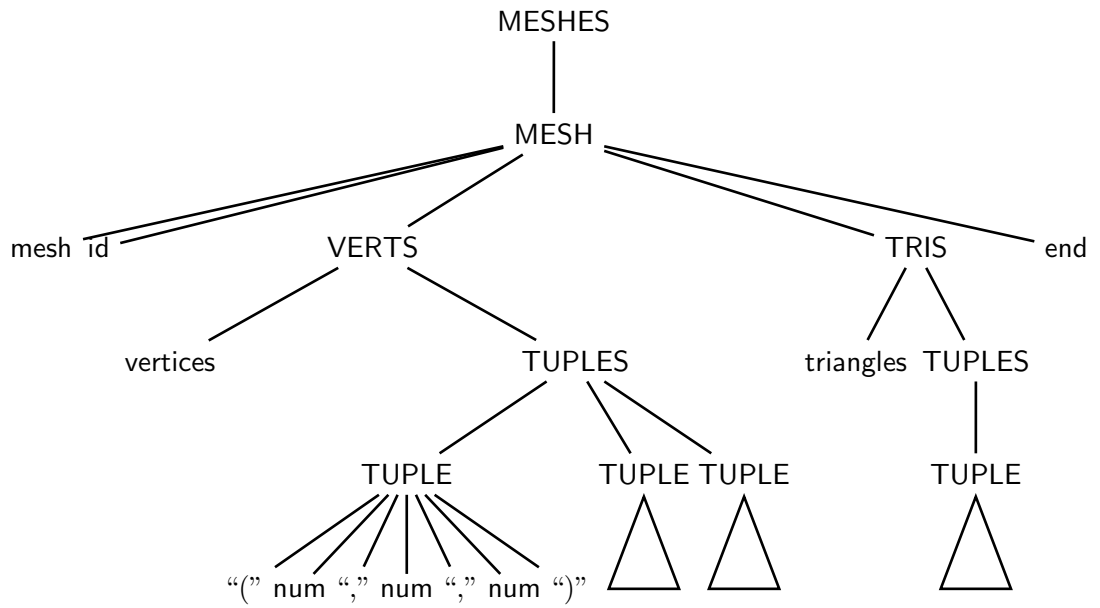
Solution

(a) Here is a viable non-ambiguous grammar (axiom MESHES):

$$\left\{ \begin{array}{l} \langle \text{MESHES} \rangle \rightarrow \langle \text{MESH} \rangle^+ \\ \hline \langle \text{MESH} \rangle \rightarrow \text{mesh id } \langle \text{VERTS} \rangle \langle \text{TRIS} \rangle \text{ end} \\ \hline \langle \text{VERTS} \rangle \rightarrow \text{vertices } \langle \text{TUPLES} \rangle \\ \langle \text{TRIS} \rangle \rightarrow \text{triangles } \langle \text{TUPLES} \rangle \\ \hline \langle \text{TUPLES} \rangle \rightarrow \langle \text{TUPLE} \rangle^+ \\ \hline \langle \text{TUPLE} \rangle \rightarrow "(" \text{ num } "," \text{ num } "," \text{ num } ")" \end{array} \right.$$

This grammar is also ELL (1). Of course it is possible to create multiple variations of the same grammar by using more or less rules than what is shown here. The language is actually regular, so a one-rule EBNF grammar is sufficient to describe it (at the cost of repeating the definition of a tuple twice).

(b) Here is the syntax tree of the sample string:



The expansion of the TUPLE nonterminal is shown only once for simplicity.

(c) A viable modified grammar is the following (modified or new rules in red):

$$\left\{ \begin{array}{l} \langle \text{MESHES} \rangle \rightarrow \langle \text{MESH} \rangle^+ \\ \hline \langle \text{MESH} \rangle \rightarrow \text{mesh id } \langle \text{VERTS} \rangle \langle \text{TRIS} \rangle [\langle \text{SUBMESH} \rangle] \text{ end} \\ \hline \langle \text{VERTS} \rangle \rightarrow \text{vertices } \langle \text{TUPLES} \rangle \\ \langle \text{TRIS} \rangle \rightarrow \text{triangles } \langle \text{TUPLES} \rangle \\ \hline \langle \text{SUBMESH} \rangle \rightarrow \text{submesh } \langle \text{MESHES} \rangle \\ \hline \langle \text{TUPLES} \rangle \rightarrow \langle \text{TUPLE} \rangle^+ \\ \hline \langle \text{TUPLE} \rangle \rightarrow "(" \text{ num } "," \text{ num } "," \text{ num } ")" \end{array} \right.$$

2. Consider the abstract syntactic support G shown below, over the terminal alphabet $\Sigma = \{i, n\}$ and nonterminal alphabet $V = \{S, A, B\}$, defined by these rules (axiom S):

$$G \left\{ \begin{array}{l} 1: S \rightarrow A B \\ 2: A \rightarrow i A \\ 3: A \rightarrow i \\ 4: B \rightarrow n B \\ 5: B \rightarrow n \end{array} \right.$$

The nonterminal A is a list of variable names, the nonterminal B is a list of numeric variable references, the terminal i represents an identifier, and the terminal n represents an integer.

We want to define an attribute grammar over the support G to perform a validity check of the variable references in B with respect to the amount of variables actually declared in A . For instance, if there are 5 variable declarations, and B contains a numeric reference greater than or equal to 5, then that reference is invalid. The same holds for any negative reference.

To this end, we define the following attributes:

<i>type</i>	<i>name</i>	<i>domain</i>	<i>(non)term.</i>	<i>meaning</i>
left	v	boolean	S, B	<i>valid</i> : true if a list of references (B) or the sentence (S) is valid
left	n	integer	A	<i>number</i> : the number of declarations in the list
right	m	integer	B	<i>maximum</i> : one past the maximum allowed variable reference

Answer the following questions (use the tables / trees / spaces on the next pages):

- Write the semantic functions for the three attributes v , n and m above.
- Decorate the syntax tree of the string:

$i \ i \ i \ 0 \ 5 \ 1 \ 2$

with the value of the attributes. Use the syntax tree prepared on the next page.

- Determine if the attribute grammar is of type one-sweep and if it satisfies the L condition, and reasonably justify your answers.
- Consider a modified grammar where rule 1 is rewritten as: $S \rightarrow B A$. Explain if the new grammar still satisfies the L condition and reasonably justify your answers.

attribute grammar to write - question (a)
(for clarity the terminals are also indexed)

#	<i>syntax</i>	<i>semantics</i>
---	---------------	------------------

1:	$S_0 \rightarrow A_1 B_2$	
----	---------------------------	--

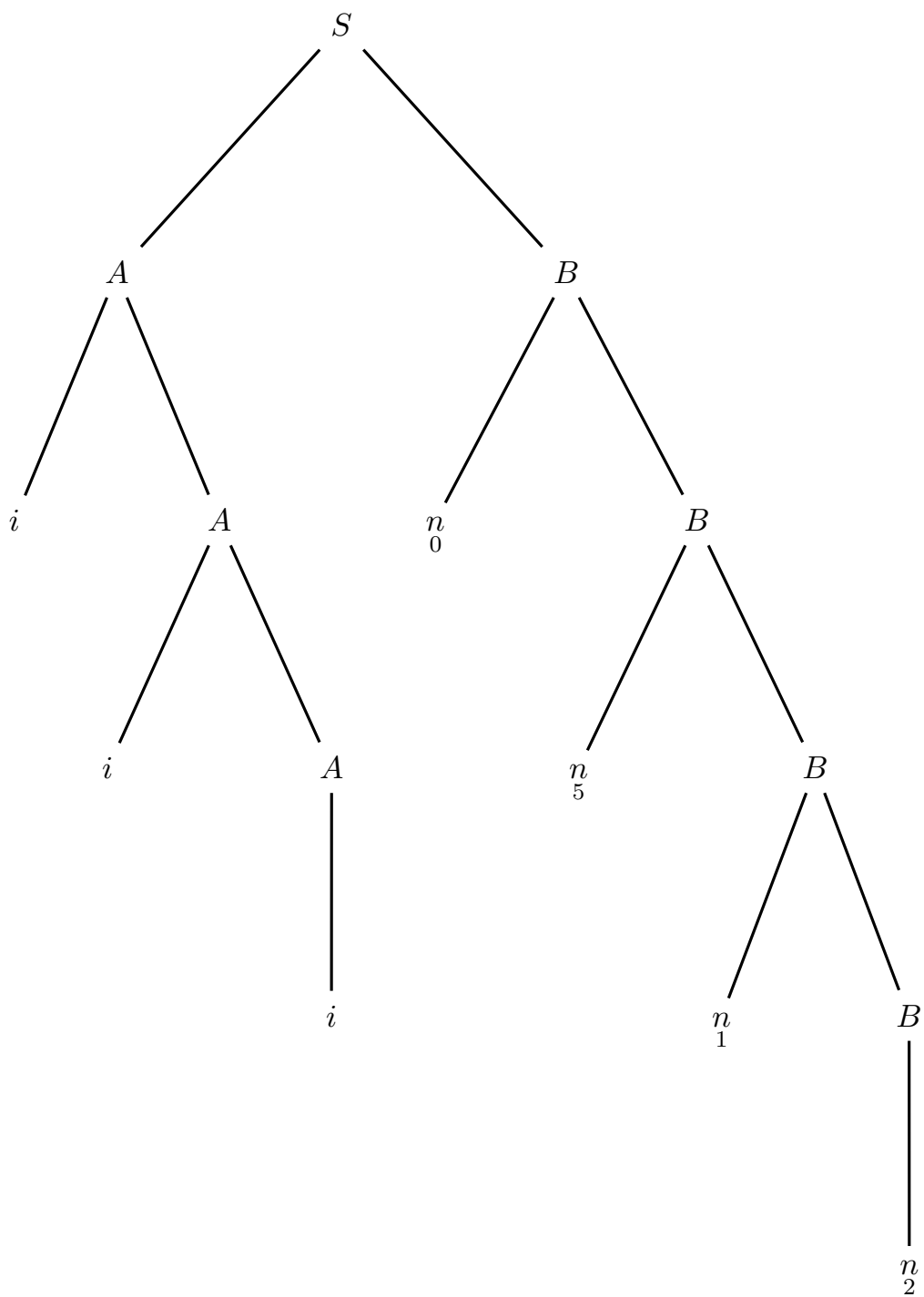
2:	$A_0 \rightarrow i_1 A_2$	
----	---------------------------	--

3:	$A_0 \rightarrow i_1$	
----	-----------------------	--

4:	$B_0 \rightarrow n_1 B_2$	
----	---------------------------	--

5:	$B_0 \rightarrow n_1$	
----	-----------------------	--

syntax tree to decorate – question (b)



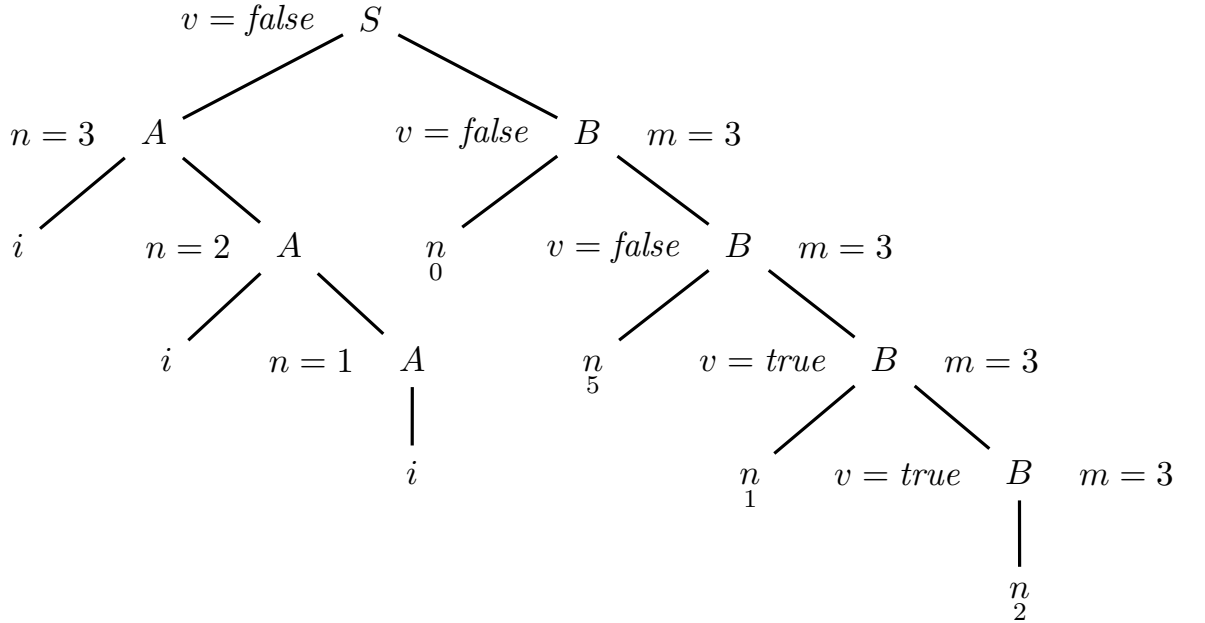
space for answering questions (c, d)

Solution

- (a) Here is the requested attribute grammar, which uses all and only the proposed attributes:

#	<i>syntax</i>	<i>semantics</i>
1:	$S_0 \rightarrow A_1 B_2$	$m_2 := n_1$ $v_0 := v_2$
2:	$A_0 \rightarrow i_1 A_2$	$n_0 := n_2 + 1$
3:	$A_0 \rightarrow p_1$	$n_0 := 1$
4:	$B_0 \rightarrow n_1 B_2$	$m_2 := m_0$ $v_0 := v_2 \wedge 0 \leq n_1 < m_0$
5:	$B_0 \rightarrow n_1$	$v_0 := 0 \leq n_1 < m_0$

- (b) Here is the decorated syntax tree:



As customary, the left or right attributes are placed on the left or right, respectively, of the nonterminal node they are associated with.

- (c) Grammar G is either ELL (2) or ELL (1), because the alternative rules 4 and 5 can be written in extended form either as $B \rightarrow n B \mid n$, which is ELL (2), or $B \rightarrow n [B]$, which is ELL (1); the square brackets indicate optionality. In all cases, by using an appropriate look-ahead depth, the grammar is definitely ELL.

The attribute grammar is one-sweep, as it can be computed in a single visit of the syntax tree. In particular, attribute n is computed bottom-up when finishing the visit of the A branch of the tree, then attribute m is propagated top-down while visiting the B branch, and when returning from that visit, attribute v is computed bottom-up. The final result is available in the tree root node.

Anyway, this is not sufficient for validating the L condition, as the requirement of the sibling visit order has to be also satisfied. Rule 1 has a non-trivial sibling dependency graph with two nodes, while all the other rules have a trivial one. The sibling graph of rule 1 is $\textcircled{A} \xrightarrow{n_1 \rightarrow m_2} \textcircled{B}$ and does not contain any rightward arrow. Therefore, the attribute grammar fully satisfies the L condition. An integrated syntactic-semantic recursive-descent analyzer for the grammar can be designed.

- (d) When swapping the nonterminals A and B in rule 1, we violate the required condition on the sibling graph, the grammar is still one-sweep but the L condition is not satisfied anymore.

4 Bonus question (5 points)

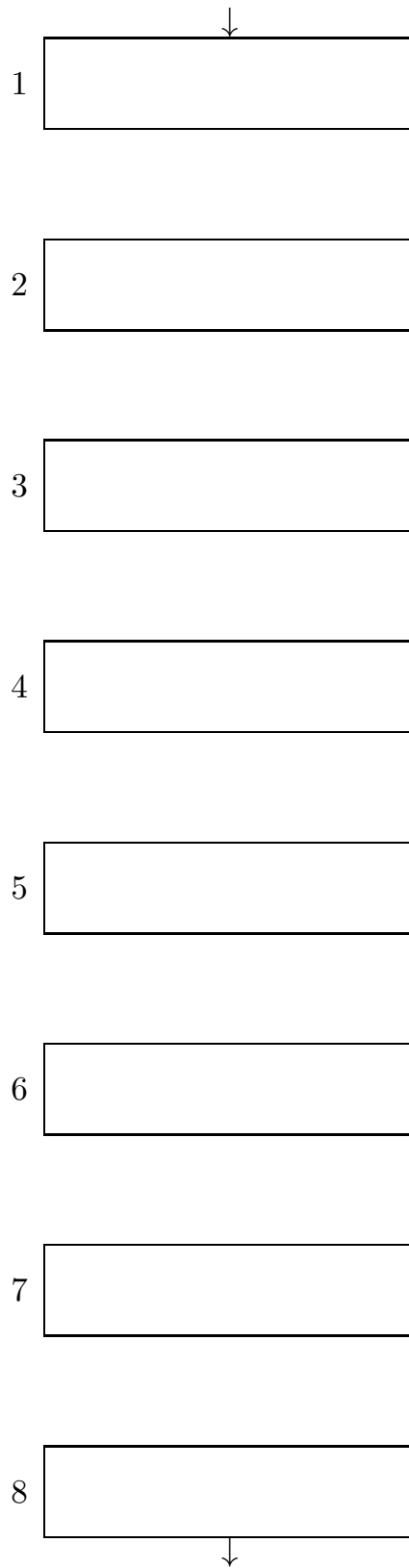
1. Consider the following short program written in pseudo-code, with three integer variables a , b and c :

```
do
  | read( $a$ )
  |  $b := b + a$ 
while  $a > 0$ 
 $c := 0$ 
while  $b > 0$  do
  |  $c := c + 1$ 
  |  $b := b - 5$ 
write( $c$ )
```

Answer the following questions (use the figures / tables / spaces on the next pages):

- (a) Translate the program to a control flow graph, where each node contains at most one statement or one expression.
 - (b) Annotate the nodes of the control flow graph with the liveness of the variables a , b and c , using your intuition.
 - (c) Write the definition of the equations for computing the liveness *in* and *out* of the node corresponding to the condition check $b > 0$, including the values of the *use* and *def* sets for that node.
 - (d) Are there any variables that are used without being initialized? How can this be understood from the liveness information?
-

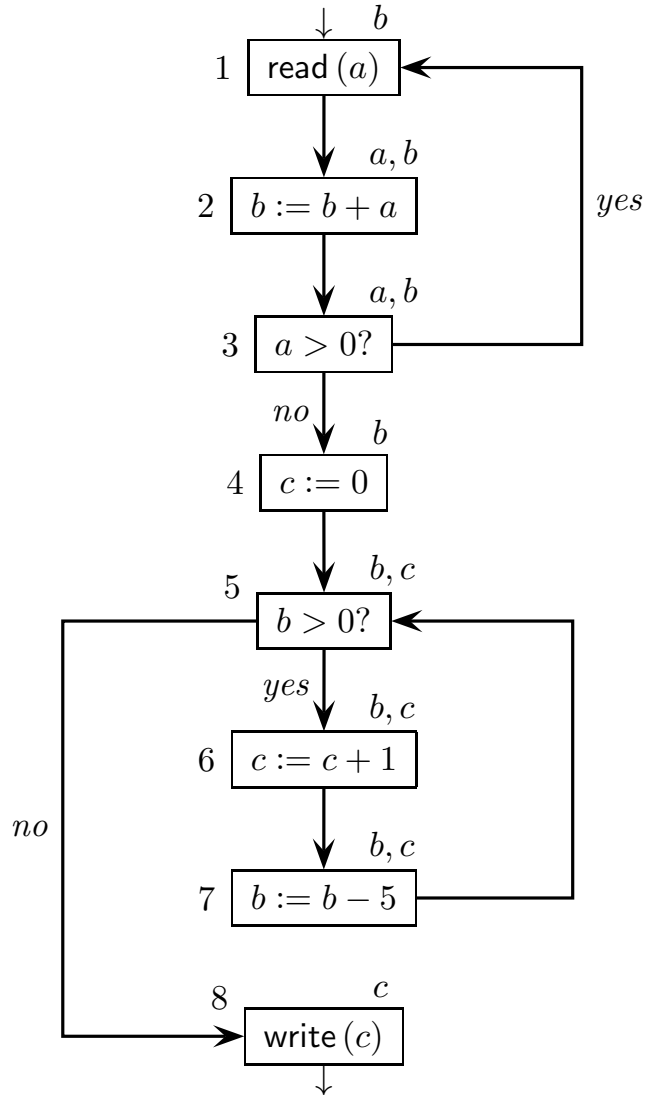
control flow graph to be completed for questions (a, b)



space for answering questions (c, d)

Solution

(a) A straightforward control flow graph representing the given program is the following:



(b) See the diagram above for the liveness annotations at the node inputs.

(c) The use and def set definitions are:

$$use(5) = \{b\} \quad def(5) = \emptyset$$

therefore the in and out equations are:

$$\begin{aligned} in(5) &= use(5) \cup (out(5) \setminus def(5)) = use(5) \cup out(5) = \{b\} \cup out(5) \\ out(5) &= in(6) \cup in(8) \end{aligned}$$

The equations are consistent with the liveness information shown above at point (b), since by filling in the solution:

$$\{b, c\} = in(5) = \{b\} \cup out(5) = \{b\} \cup in(6) \cup in(8) = \{b\} \cup \{b, c\} \cup \{c\} = \{b, c\}$$

the two members of the in equation are equal. Notice that at the input of node 8 only variable c is live, because variable b is not live in to node 8.

(d) Variable b is used in node 2 without being initialized. This is visible from the fact that b is live in from the entry node (node 1).